

Lec 07 - Miscellaneous

COMPSCI 220 - Perretta

Exercise 0, Part 1 (5 min)

Implement a function `min_element` and refactor the code below to use it.

`min_element` requires that the input array has at least one element.

```
const numbers = [
  6, 3, 8, 2, 9, 7, 2
];
let min = numbers[0];
for (const num of numbers) {
  if (num < min) {
    min = num;
  }
}
console.log(min);
```

2

```
const dates = [
  "2026-02-20", "2026-02-23", "2026-02-21"
];
let earliest = dates[0];
for (const date of dates) {
  if (date.localeCompare(earliest) < 0) {
    earliest = date;
  }
}
console.log(earliest);
```

2026-02-20

Exercise 0, Part 2 (10 min)

```
// Returns an array containing the earliest submission from each user.  
// The form submissions are not sorted.  
// If a user has no submissions, the returned array should contain undefined.  
function earliest_submissions(users: string[], submissions: Submission[]) {}  
  
class Submission {  
  constructor(public readonly user: string, public readonly timestamp: string) {}  
}  
  
const submissions = [  
  new Submission("WALUIGI", "2026-02-24"), new Submission("WALUIGI", "2026-02-21"),  
  new Submission("sanic", "2026-01-31"), new Submission("sanic", "2026-02-20"),  
];  
  
const earliest_subs = earliest_submissions(  
  ["sanic", "knuckles", "WALUIGI"], submissions);  
console.log(earliest_subs);  
// '2026-01-31' for sanic, undefined for knuckles, '2026-02-21' for WALUIGI
```

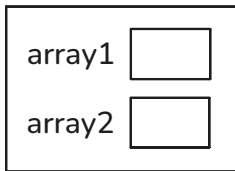
Exercise 1 (5 min)

Draw & update the memory diagram for each statement specified.

Use reference semantics for arrays and objects, value semantics for numbers.

```
class Parrot {  
  constructor(public age: number) {}  
}  
  
function main() {  
  // Draw memory diagram after this statement.  
  const array1 = [new Parrot(42), new Parrot(43)];  
  
  // Update memory diagram after the loop exits.  
  const array2 = [];  
  for (const parrot of array1) {  
    array2.push(parrot);  
  }  
}
```

main



Exercise 2 (7 min)

What is the output of this code? Draw the memory diagram to help your answer.

Hint: `===` in JS compares references for array and object types

```
class Parrot {  
  constructor(public age: number) {}  
}  
  
const array1 = [  
  new Parrot(42), new Parrot(65)  
];  
const array2 = array1.filter(  
  (parrot) => parrot.age > 42);  
const array3 = array2.map(  
  (parrot) => new Parrot(parrot.age));  
const array4 = array3;  
array2[0].age++;
```

```
console.log(array1 === array2);  
console.log(array2 === array3);  
console.log(array3 === array4);  
  
console.log(array1[1] === array2[0]);  
console.log(array3[0] === array2[0]);  
  
console.log(array1[1].age);  
console.log(array2[0].age);  
console.log(array3[0].age);  
console.log(array4[0].age);
```

Exercise 3 (7 min)

List the input categories for `earliest_submissions` that a test suite should cover.

```
// Returns an array containing the earliest submission from each user.  
// The form submissions are not sorted.  
// If a user has no submissions, the returned array should contain undefined.  
function earliest_submissions(users: string[], submissions: Submission[]) { ... }
```